

1. How do you implement Singleton in Java?

A Singleton ensures that only one instance of a class can exist in JVM and provides a global access to that instance.

To implement a Singleton, I usually make the constructor private so external users cannot create objects directly.

I store the instance in private static variable.

I provide a public static method to return the instance.

There are two common approaches: Eager loading & Lazy loading

In lazy loading, the object is not created until it is actually needed, and a singleton always use lazy loading

But lazy loading won't be safe if there are two threads `getInstance()` at the same time. So we need to

use the double null approach. This approach is also called Double-Checked Locking.

△ The first null avoids unnecessary synchronization after the object has been created.

△ The second is to ensure only one thread creates the object.

△ Volatile prevents instruction reordering & make threads visible.

2. Prevent Memory leak.

Memory leak happens when objects are no longer needed but are still referenced. This will prevent the GC from reclaiming them.

Common causes includes unclosed IO resource,

ThreadLocal values not being removed,

Static collections that keep growing,
improperly configured thread pools, and

custom cache implementations without eviction policies.

To avoid memory leaks, I use `try-with-resource` to close resource, call `remove()` on ThreadLocal variables

avoid unbounded static collections, configure thread pool carefully, release locks in finally blocks,

and use cache libraries such as Caffeine or Guava instead of maintaining large static map manually.

3. Why we need customized exception?

There are mainly two types of exceptions in Java programming:

△ Checked Exceptions, also called Compile time exception, which requires developers to handle it during compile time. Developers usually use "try-catch-finally" block to handle it.

△ Unchecked Exceptions, also called Runtime exception, does not necessarily require developer to handle it

during Runtime. It occurs in runtime.

We need customized exceptions when built-in exceptions are not sufficient to satisfy the business needs. For example, instead of throwing a Runtime exception, we can create exceptions such as `InsufficientBalanceException`, `InvalidOrderException`. This will make the code readable & easy to maintain.

In Java, I usually use checked exceptions for recoverable situations such as missing file or unavailable resource. And the developer is expected to handle the error.

For invalid user input or unexpected business validation failures, I use runtime exceptions because user behavior cannot always be predicted. Forcing developer to handle them is cumbersome.

4. What is constant String pool?

String Constant Pool is a special memory area used to store String literals. It allows Java to reuse String objects instead of creating duplicated objects. For example, if two variables are assigned the same String literal, they will reference the same objects in String constant pool. String constant pool is stored in Heap. String is immutable in Java. Once a String is created, its value cannot be changed, and the object cannot be modified. Thus, Java can safely share the same String object from the String constant pool without worrying that one reference will affect another.

5. What is the diff between Interface & abstract Class.

Both interface and abstract class are used to achieve abstraction in Java. An abstract class can contain both abstract methods and concrete methods. It can also have instance variables and constructors. It is typically used when multiple closely related classes need to share common state & implementation.

A interface mainly defines a contract or behavior that classes have to follow. A class can implement multiple interfaces, which allows multiple inheritance of type. In contrast, a class can only extend only one abstract class or concrete class in Java.

Since Java 8, interfaces can also contain default methods and static methods. In addition, functional Interfaces are widely used with lambda expression and Stream API.

I usually use abstract class when I want to share code among related classes. I choose interface when I want my subclasses follow different behavior from multiple interfaces.

6. How to avoid race condition?

A race condition occurs when multiple threads access the same shared resource at the same time. This leads to unpredictable results.

To avoid it, I use two methods: Using lock mechanism & non-locking mechanism.

Δ. locking mechanism includes synchronized or. ReentrantLock, which allows only one thread to access the critical section (shared resource) at a time.

Δ. Non-locking mechanism. uses. Atomic Integer or. Atomic Reference. These classes use CAS (Compare-And-Swap) to perform thread safe updates without using locks.

Δ. Immutable class & thread safe Collections, such as. ConcurrentHashMap can also help to reduce the race conditions.

7. Difference between wait() & sleep().

Both wait() & sleep() temporarily pause a thread, but they are used differently.

Sleep() is a static method of the Thread class, while wait() is a method of Object class.

Sleep() does not release any lock that thread holds, while wait() releases the lock and allows other thread to enter the synchronized block.

wait() must be called inside a synchronized block or synchronized method. It is usually used for thread communication together with notify() or notifyAll().

sleep() is mainly used to pause execution for some time.

8. Java 8, 11, 17, 21

Java 8: The Functional Revolution.

1. Functional Interface: Enabled writing functional-style code and passing blocks of code as arguments.

2. Stream API: Provide a declarative and pipeline-based approach to processing collections of objects.

3. Optional Class: Introduced a wrapper to handle the null values and avoid Null Pointer Exceptions.

- 4) Date and Time API: Replaced notoriously flawed "java.util.Date" and "Calendar" with a clean, thread-safe API (LocalDate, LocalTime, ZonedDateTime)
- 5) Interface Improvements: Allowed for default keyword & static method in interfaces.

Java 11: The Enterprise Cleanup:

Java 11 brought API modernizations, removed outdated legacy modules (CORBA and Java EE), Java 11 established a stable enterprise baseline.

- 1) Local variable Syntax: "var." allowed for Lambda Expressions parameters, cleaning code.
- 2) HTTP Client: Added a standardized, modern API for sending HTTP requests and handling responses, replacing older "HttpURLConnections"
- 3) String Methods: Introduced convenient new methods like "isBlank()", "lines()", "repeat()" and "strip()"
- 4) Running Source Files: Ability to execute a java file directly with "java" or "filename.java". Do not need separate compilation step.

Java 17

Java 17 significantly clean up language syntax, reduce Boilerplate, added developer-friendly ways to control class hierarchy.

- 1) Record: A concise way to create an immutable class. Auto-generating constructors, equals(), hashCode() & toString() [override].
- 2) Sealed Class: Provide precise control over which classes or interfaces can extend or implement them, improving domain modeling.
- 3) Pattern Matching for "instanceof": Eliminated the need for manual type casting after checking the object's type.
- 4) Switch Expressions: Modernized "switch" statements, allowing them to return values, yield results, and avoid falling through cases without breaking statements.

Java 21: The Concurrency Era.

Java 21 represents a massive leap in how Java handles both readability and high-throughput concurrency.

1) Virtual Threads: Introduced. Lightweight, JVM-managed threads (Project Loom). This allows developers to run millions of concurrent tasks with blocking I/O without the heavy memory overhead of traditional OS threads.

2) Pattern Matching for "Switch": Extended pattern matching to "switch" cases, allowing direct type checks and conditional evaluations (guards) within the switch block.

3) Record Patterns: Enable the automatic extraction and destructuring of data directly inside patterns, vastly simplifying data extraction.

4) Sequenced Collections: Introduced unified Interfaces ["SequencedCollection", "SequencedMap", "SequencedSet"] with standard methods for operating on the first element, last element, or reverse order.